Understanding ML, Chapter 4: Discriminative and Generative Classification

After reading this you can say, for any classifier, what it models, what it assumes, what knob you are turning, what experiment reveals the difference, and what goes wrong when the assumption is false

Fundamentals of Machine Learning, midterm

2026-11-03

What this chapter is about, in one paragraph

Chapter 2 settled what to do once the posterior $p(y = c \mid x)$ is known: pick the class with the largest posterior, or compare it with a cost threshold. Chapter 4 is about the only hard part, *getting the posterior out of labelled data*. There are exactly two ways. The **discriminative** route writes a parametric family $p_\theta(y \mid x)$ (a score, squashed by a sigmoid or a softmax) and fits it to predict the labels; it learns *where the label changes* and never looks at how the inputs are distributed. The **generative** route writes a story of how a labelled example is produced (choose a class from the prior π_c, then draw features from the class-conditional $p_\theta(x \mid y = c)$), fits that story, and reverses it with Bayes' rule; it learns *what each class looks like*. Everything else in the chapter is a consequence of that choice: which assumptions you make (Gaussian clouds, shared or separate covariance, independent features), how many numbers you estimate, how much data you need, what the boundary looks like, what happens when the class frequencies change between training and deployment, and whether the model can say "I have never seen anything like this". The quantities we manipulate in the experiments are the amount of labelled data N, the covariance assumption, the smoothing constants λ and α, the class prior, the decision threshold, and the familiarity of the input.

Contents

1 The decision is known, the posterior is not: two routes **2**

2 The discriminative route: scores, sigmoid, softmax, cross-entropy **4**

3 The generative route: a story, reversed by Bayes' rule **8**

4 Gaussian class models: LDA and QDA **9**

5 Naive Bayes: independence given the class **13**

6 Comparing the routes: the data-size experiment **17**

7 Calibration, class imbalance and prior shift **18**

8 Unfamiliar inputs, combined systems, language models **21**

Coverage of the textbook chapter **23**

1 The decision is known, the posterior is not: two routes

Posterior, prior, class-conditional, and the decision that uses them

Let $Y \in \{1, \ldots, C\}$ be the class and $X \in \mathbb{R}^d$ the feature vector.

- The **posterior** $p(y = c \mid x)$: probability of class c *once* x *has been seen.*
- The **prior** $\pi_c = p(y = c)$: how frequent class c is *before* any input is seen.
- The **class-conditional** $p(x \mid y = c)$: the distribution of inputs that class c produces (a density for continuous x, a mass function for discrete x).

Given the posterior, the decision is a one-liner: under zero–one loss $f^*(x) = \arg\max_c p(y = c \mid x)$ [on your sheet]; with costs, alert iff $p(y = 1 \mid x) > C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet]. The posterior is unknown; it must be *estimated* from $\mathcal{D}_N = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$.

Purpose. The whole chapter is the estimation problem; the decision rule is inherited from Chapter 2.

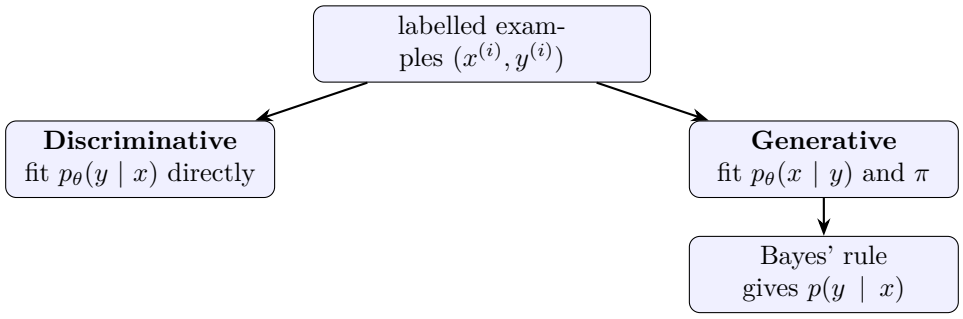
Impact. A better posterior estimate improves *every* policy built on it; a different cost threshold changes the action without changing the estimate.

Discriminative and generative classifiers

- A **discriminative** model directly approximates the posterior with a parametric family $p_\theta(y = c \mid x)$. Training produces $p_{\hat{\theta}}(y = c \mid x)$. It learns *where the label changes* as a function of the features.
- A **generative** classifier models the prior $p(y = c)$ *and* the class-conditional $p(x \mid y = c)$, then obtains the posterior by Bayes' rule. It learns *what each class looks like* and asks which class explains the observed input best.

Purpose. Same target (a posterior), two opposite directions of modelling.

Impact. Discriminative: fewer assumptions, nothing about the inputs, needs more data. Generative: more structure, works with little data if the structure is right, carries a persistent error if it is wrong, and gains two abilities the other route lacks (plausibility of an input, handling missing features).



Where each route is used

- *Spam filtering.* Naive Bayes (generative) models token frequencies of spam and of legitimate mail; cheap, estimated from counts, still a baseline or a first triage layer.
- *Fraud and intrusion screening.* Class-conditional models score how *compatible* an event is with normal behaviour and with known attacks; they serve classification and anomaly triage at once. Modern systems add discriminative and behavioural signals.
- *Image classification.* Deep discriminative networks dominate: modelling the full density of

images per class is far harder than learning where the label changes.

"Generative" is about the *direction* of modelling, not the task: a class-conditional model can infer the class of an observed x or, in principle, draw a new x for a chosen class. Classical generative classifiers and modern generative AI share the direction; families and scales differ enormously.

Discriminative versus generative, the one table to remember

Question	Discriminative	Generative
Modelled quantity	posterior $p_\theta(y \mid x)$ or a boundary	prior π_c and class model $p_\theta(x \mid y = c)$
Assumptions on the input	usually fewer	usually stronger (a family for each class)
What is fitted	one conditional objective (labels given inputs)	class counts, then one model per class from that class's examples
What it can do	rank the known classes	rank, plus judge plausibility, sample inputs, marginalise missing features
Failure mode	underdetermined with little data; confident about unfamiliar inputs	persistent bias when the family is wrong; double counting under false independence
When to prefer	many labels, raw high-dimensional input, prediction is the only goal	few labels, credible class model, need for a reject option or prior correction

Key insight. Both routes end in a softmax of class scores. A discriminative model fits the scores directly; a generative model builds them as $\log \pi_c + \log p_\theta(x \mid y = c)$. Everything that separates the routes is where the scores come from and what had to be assumed to get them.

How the professor asks it

Q. *Explain why a posterior estimate $\hat{p}(y = 1 \mid x)$ and the alert decision are not the same object, and which of the two changes when (a) the costs change, (b) the model is retrained.*
A. They are different objects: the posterior is a number in $(0, 1)$ produced by the model; the decision is an action obtained by comparing that number with a threshold fixed by costs. (a) Changing $C_{\text{FP}}, C_{\text{FN}}$ moves the threshold $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet] and so the action, without touching the model; the same 0.35 is "benign" at $t = 0.5$ and "attack" at $t = 0.2$. (b) Retraining changes the posterior, not the costs. One probability model supports several policies; no policy repairs a bad probability.

Q. *A security team says "attacks are only 1% of traffic, so a flow should almost never be classified as an attack". Give the generative argument against this, with the numbers $\pi = (0.99, 0.01)$, $p(x \mid \textit{benign}) = 0.001$, $p(x \mid \textit{attack}) = 0.20$.*
A. The prior is only one factor of the posterior odds; the likelihood ratio is the other. Prior-weighted likelihoods: $0.99 \times 0.001 = 0.00099$ and $0.01 \times 0.20 = 0.00200$; posterior of attack $0.002/0.00299 = 0.67$. In odds: prior odds $1/99$ times likelihood ratio 200 equals $2.02 > 1$. The prior settles the matter only when the input carries no evidence (likelihood ratio 1); here the flow is 200 times more compatible with the attack model, which beats the 99-to-1 prior.

2 The discriminative route: scores, sigmoid, softmax, cross-entropy

Score (logit), sigmoid, logistic regression

A discriminative model usually first produces a real-valued **score** $s_{\theta,c}(x)$ per class, then turns scores into probabilities, then into an action:

$$\{s_{\hat{\theta},c}(x)\} \longrightarrow \{p_{\hat{\theta}}(y = c \mid x)\} \longrightarrow \hat{y}(x).$$

The score model fixes how inputs are *ranked*; the probability map quantifies *relative* uncertainty among classes; the decision loss picks the *action*. For two classes one score suffices and the **sigmoid** converts it, $p_\theta(y = 1 \mid x) = \sigma(s_\theta(x)) = 1/(1 + e^{-s_\theta(x)})$ [on your sheet]. **Logistic regression** is the linear score $s_\theta(x) = w^\top x + b$. Sigmoid facts you must be able to use: $0 < \sigma < 1$; $\sigma(-s) = 1 - \sigma(s)$ [on your sheet]; strictly increasing; $\sigma(s) > t \iff s > \log \frac{t}{1-t}$ [on your sheet]; $\sigma' = \sigma(1 - \sigma)$ [on your sheet]. Values: $\sigma(0) = 0.5$, $\sigma(0.5) = 0.62$, $\sigma(2) = 0.88$, $\sigma(-2) = 0.12$, $\sigma(5) = 0.993$.

Purpose. Turn an unbounded number, convenient to model (a linear function, a network output), into a probability.

Impact. A score of 0 means "no idea"; large positive, "confidently class 1". With $w = (1, -2)$, $b = 0.5$, $x = (2, 1)$: $s = 2 - 2 + 0.5 = 0.5$, $p = \sigma(0.5) \approx 0.62$. One unit more of x_j adds w_j to the log-odds, i.e. multiplies the odds by e^{w_j}, whatever the other features are.

The log-odds are the score; the boundary is a hyperplane; costs only slide it

Step 1 (log-odds). From $p = \sigma(s)$ and $1 - \sigma(s) = \sigma(-s)$: $\dfrac{p}{1-p} = \dfrac{1/(1+e^{-s})}{e^{-s}/(1+e^{-s})} = e^s$, hence $\log \dfrac{p}{1-p} = s_\theta(x) = w^\top x + b$ [on your sheet]. Logistic regression is the model "log-odds linear in x".

Step 2 (equal costs). Zero–one Bayes decision: declare 1 iff $p > \frac{1}{2}$. Since σ is increasing and $\sigma(0) = \frac{1}{2}$, this is $s > 0$. For a linear score, $\{x : w^\top x + b = 0\}$ is a hyperplane.

Step 3 (unequal costs). The cost threshold on the probability is $t = C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$ [on your sheet]. Invert the sigmoid: $\sigma(s) > t \iff 1 + e^{-s} < 1/t \iff e^{-s} < (1-t)/t \iff s > \log \frac{t}{1-t}$. With $\frac{t}{1-t} = \frac{C_{\text{FP}}}{C_{\text{FN}}}$ the rule is

$$\hat{y} = 1 \iff s_\theta(x) > \log \frac{C_{\text{FP}}}{C_{\text{FN}}} \quad \text{[on your sheet]}.$$

Reading. The boundary moves to a *parallel* hyperplane $w^\top x + b = \log(C_{\text{FP}}/C_{\text{FN}})$. The score family fixes the geometry (linear score, flat boundary; curved score g_θ, curved surface $g_\theta(x) = 0$); the costs only translate it along the score axis. Cyberattack with $C_{\text{FN}} = 10$, $C_{\text{FP}} = 1$: threshold $s > \log 0.1 = -2.30$, i.e. alarm already when the score is mildly negative (probability above $\sigma(-2.30) = 0.09$). Expensive misses lower the threshold; expensive false alarms raise it.

Sliding the threshold: one model, many policies

Setup. Fit one logistic regression on intrusion flows (training rows). Keep a validation set with labels.

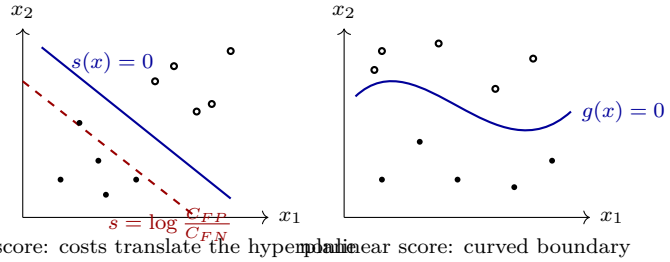
Held fixed. The fitted w, b; the validation rows.

What we manipulate. The decision threshold on the score, $\tau = \log(C_{\text{FP}}/C_{\text{FN}})$, from -4 to $+4$ (equivalently the cost ratio from 1:55 to 55:1).

What we measure. At each τ: recall of attacks, false-positive rate, number of alarms per 100,000 benign flows.

What the plot shows. Recall rises and the false-positive rate rises as τ falls; the curve of (false-positive rate, recall) is the ROC curve of *this* model. The model's ranking never changes; only the cut point moves.

How to read it. If two thresholds give very different recall at the same false-positive rate, you misread the plot: one model has one curve. To get a *better* curve you must change the model (features, family), not the threshold. Pick τ on validation from the costs, then report on the untouched test set.



linear score: costs translate the hyperplane nonlinear score: curved boundary

Softmax for C classes

$p_\theta(y = c \mid x) = \text{softmax}(s_\theta(x))_c = e^{s_{\theta,c}(x)} / \sum_{k=1}^{C} e^{s_{\theta,k}(x)}$ [on your sheet]. Exponentiate to make every score positive, divide by the total so the classes compete for probability one.

Purpose. The C-class version of the sigmoid.

Impact. Three facts. (i) The ordering of probabilities is the ordering of scores, so $\arg\max_c p = \arg\max_c s_c$. (ii) **Shift invariance**: adding the same constant a to every score multiplies numerator and denominator by e^a; nothing changes, so only $C - 1$ score *contrasts* are identifiable (and by hand you subtract the largest score before exponentiating, which avoids huge numbers). (iii) With $C = 2$, $\frac{e^{s_1}}{e^{s_1}+e^{s_2}} = \frac{1}{1+e^{s_2-s_1}} = \sigma(s_1 - s_2)$ [on your sheet]: softmax is the sigmoid of the score difference. Scores $(2, 0, -1)$: exponentials $(7.39, 1.00, 0.37)$, total 8.76, probabilities $(0.84, 0.11, 0.04)$. Implementations keep C scores rather than $C - 1$ so that every class plays the same role and one output layer serves any C; a reference class removes the redundancy when coefficients must be interpreted.

Cross-entropy, and why it is maximum likelihood

For example i, $\ell_{\text{CE}}^{(i)}(\theta) = -\sum_c \mathbf{1}(y^{(i)} = c) \log p_\theta(y = c \mid x^{(i)}) = -\log p_\theta(y^{(i)} \mid x^{(i)})$ [on your sheet]: minus the log of the probability given to the *true* class. True-class probabilities $0.9, 0.5, 0.1$ cost $0.105, 0.693, 2.303$ (natural logs): confident mistakes are punished hardest.

Purpose. The training loss for any probabilistic classifier, from logistic regression to deep networks.

Impact. Minimising the empirical cross-entropy is *exactly* maximising the conditional likelihood of the labels given the inputs; the model is never asked to explain the inputs.

Cross-entropy equals negative conditional log-likelihood

Assume the labels are conditionally independent given their inputs. The conditional likelihood is the product $L_{\text{cond}}(\theta) = \prod_{i=1}^{N} p_\theta(y^{(i)} \mid x^{(i)})$ (independence turns a joint probability into a product). Take $-\log$: the product becomes a sum,

$$-\log L_{\text{cond}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(y^{(i)} \mid x^{(i)}) = N\, \widehat{R}_{\text{CE}}(\theta) \quad \text{[on your sheet]}.$$

Since $-\log$ is decreasing, maximising L_{cond} and minimising $\widehat{R}_{\text{CE}}$ have the same solutions. Three examples with true-class probabilities $0.9, 0.5, 0.1$: likelihood $0.9 \times 0.5 \times 0.1 = 0.045$; negative log $0.105 + 0.693 + 2.303 = 3.10$; empirical cross-entropy $3.10/3 = 1.03$. "Conditional" is the whole point: only the labels given the inputs are explained. For logistic regression the gradient is $\frac{1}{N}\sum_i(\hat{p}^{(i)} - y^{(i)})x^{(i)}$ [on your sheet], the average of error times input, and the objective is convex: no local minima.

Deep discriminative classifiers

A feed-forward network computes $h^{(0)} = x$, $h^{(\ell)} = \varphi(W_\ell h^{(\ell-1)} + b_\ell)$ for $\ell = 1, \ldots, L-1$ (multiply by a matrix, add a vector, bend with an elementwise nonlinearity φ), then scores $s_\theta(x) = W_L h^{(L-1)} + b_L$ and softmax. Cross-entropy trains all layers end to end.

Purpose. Let the score be a *curved* function of x so the boundary can curve.

Impact. Logistic regression is the case $L = 1$ (no hidden layer). The learned representation is optimised for predicting the label, not for describing the inputs; that is why the next concept applies to every discriminative model, deep or not.

What the discriminative objective does not provide

A discriminative classifier focuses on predicting Y from X. Consequences: it need not learn whether an input x is *plausible*; a large $\max_c p_{\hat{\theta}}(y = c \mid x)$ does not mean x resembles the training data; it has no mechanism for sampling inputs or marginalising missing features. Softmax probabilities are *relative* comparisons among the modelled classes: if all classes explain an unfamiliar input poorly, one of them can still get posterior near one.

Purpose. Know the limit of the route before trusting its confidence.

Impact. Confidence is not familiarity. This is the motivation for the reject option of Section 8.

The toaster experiment: out-of-distribution confidence

Setup. Train a three-class network (cat, dog, horse) with cross-entropy on photographs of those animals.

Held fixed. The trained network; the softmax output layer.

What we manipulate. The *familiarity* of the test input: animal photos from the test split, then inputs from outside the training classes (a toaster, random noise, a blank image).

What we measure. $\max_c \hat{p}(y = c \mid x)$, the top softmax probability, on each group.

What the plot shows. Histograms of top probability: high for familiar inputs (as expected) and *also* high for many unfamiliar ones. The toaster gets scores $(4.0, 0.5, -1.0)$, softmax $(0.96, 0.03, 0.01)$: "cat, 96%".

How to read it. The 0.96 measures how much better "cat" explains the toaster than "dog" and "horse" do, which is all a softmax can measure; nothing in the objective asked "is this even an animal?". A high top probability therefore separates the *known classes*, not familiar from unfamiliar. The experiment that fixes this adds a likelihood check (Section 8).

Reading $\max_c p(y = c \mid x)$ as "probability that the input is normal"

Tempting because the number is near one and looks like certainty. It inflates perceived reliability on novel inputs: a new attack mechanism far from every known class still receives a confident known-class label, so the detector reports high confidence precisely where it knows least. Fix: a reject option based on the class likelihood $\log p(x \mid y = c^*)$, which only a generative model provides, or a separate anomaly model.

Class imbalance treated as a bug of the discriminative route

If training and deployment share the same distribution, an imbalanced training prior is *not* an error: the fitted posterior should reflect the prevalence, and the 0.5 contour moves toward the positives (with 5% positives it sits deep in the positive region). The real difficulty is *too few minority examples* to locate the boundary; weighting or resampling helps in that small-sample regime, has limited benefit with enough minority data, and distorts the probabilities (Section 7). Fix: diagnose "few positives" (estimation) versus "prevalence changes at deployment" (shift) before touching the data.

Score versus probability versus decision

- **Score** s: unbounded, fixes the ranking and the boundary geometry; compare it with $\log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$ to decide, you never need σ.
- **Probability** $\sigma(s)$ or softmax: in $(0,1)$, relative to the modelled classes; log-odds back to score: $s = \log \frac{p}{1-p}$ ($p = 0.9$ gives $s = 2.20$).
- **Decision**: the action after a cost threshold; changing costs slides the boundary, changing the family bends it.

Key insight. The score family decides the *shape* of the boundary, the costs decide its *position*, and the softmax only compares the classes it knows.

How the professor asks it

Q. *A logistic regression for intrusion detection has $w = (1.5, -0.8)$, $b = -1$. The cost of a missed attack is ten times that of a false alarm. Derive the decision rule in terms of the score, state the geometric effect on the boundary, and decide the flow $x = (1, 1)$.*
A. Rule: alert iff $s > \log(C_{\mathrm{FP}}/C_{\mathrm{FN}}) = \log(1/10) = -2.30$. Derivation: the probability threshold is $t = 1/(1+10) = 1/11$ and $\sigma(s) > t \iff s > \log \frac{t}{1-t} = \log \frac{1}{10}$. Geometry: the boundary $w^\top x + b = -2.30$ is a hyperplane parallel to the equal-cost one, shifted toward the benign side, so more flows are flagged. Flow: $s = 1.5 - 0.8 - 1 = -0.3 > -2.30$, so alert, although $\sigma(-0.3) = 0.43 < 0.5$ would say "benign" at equal costs.

Q. *Why does adding the same constant to all C scores leave the softmax unchanged, and what does this say about how many scores are really needed?*
A. Unchanged because $e^{s_k + a} = e^a e^{s_k}$ for every k, so e^a factors out of numerator and denominator and cancels. Consequently only the $C - 1$ differences $s_c - s_C$ matter: with $C = 2$ the softmax is $\sigma(s_1 - s_2)$, one score. Implementations keep C outputs for symmetry and vectorisation; by hand, subtract the largest score first to avoid overflow, it changes nothing.

Q. *Explain why minimising cross-entropy is maximum likelihood, and why the word "conditional" is essential to the comparison with the generative route.*
A. Because $-\log \prod_i p_\theta(y^{(i)} \mid x^{(i)}) = \sum_i -\log p_\theta(y^{(i)} \mid x^{(i)}) = N \widehat{R}_{\mathrm{CE}}$ under conditional independence of the labels given the inputs; $-\log$ is decreasing, so the minimisers coincide. "Conditional" because the likelihood explains the labels given the inputs only; the generative route maximises the *joint* likelihood $\prod_i \pi_{y^{(i)}} p_\theta(x^{(i)} \mid y^{(i)})$, which also has to explain the inputs. That is the extra structure (and the extra assumptions) of the generative route in one word.

3 The generative route: a story, reversed by Bayes' rule

Generative classifier

An observation is imagined to be produced in two steps: first choose a class, then generate features from that class,

$$Y \sim \text{Categorical}(\pi), \qquad X \mid (Y = c) \sim p_\theta(\cdot \mid y = c), \qquad p_\phi(x, y = c) = \pi_c \, p_\theta(x \mid y = c),$$

with parameters $\phi = (\pi, \theta)$. The joint factorises as prior times class-conditional.

Purpose. Describe each class, then compare descriptions.

Impact. The same fitted story can classify (infer y from x), judge plausibility ($p(x \mid y)$ small for every class), fill in missing features (marginalise), and generate.

Bayes' rule for classification, and its odds form

By the definition of conditional probability, $p(y = c \mid x) = p(x, y = c)/p(x)$. Numerator: $\pi_c \, p_\theta(x \mid y = c)$ by the factorisation. Denominator: the marginal $p(x) = \sum_k p(x, y = k) = \sum_k \pi_k \, p_\theta(x \mid y = k)$, a sum over the exhaustive, exclusive classes. Hence

$$p_\phi(y = c \mid x) = \frac{\pi_c \, p_\theta(x \mid y = c)}{\sum_{k=1}^{C} \pi_k \, p_\theta(x \mid y = k)} \quad \text{[on your sheet]}.$$

The denominator does not depend on c, so the arg max is decided by the **prior-weighted likelihood** $\pi_c \, p_\theta(x \mid y = c)$: "how common is the class" times "how well does it explain this x". For two classes divide the two numerators; the denominator cancels:

$$\underbrace{\frac{p(y = 1 \mid x)}{p(y = 0 \mid x)}}_{\text{posterior odds}} = \underbrace{\frac{\pi_1}{\pi_0}}_{\text{prior odds}} \times \underbrace{\frac{p(x \mid y = 1)}{p(x \mid y = 0)}}_{\text{likelihood ratio}} \quad \text{[not on your sheet: know it]}.$$

Rare-attack numbers: benign prior 0.99, likelihood 0.001; attack prior 0.01, likelihood 0.20. Prior-weighted: 0.00099 versus 0.00200; posterior of attack $0.002/0.00299 = 0.67$. Attacks stay rare in the population, yet this flow is more compatible with the attack model, and the posterior says so.

The generative likelihood splits by class; the MLE prior is the class frequency

For independent labelled observations, $L(\phi) = \prod_i p_\phi(x^{(i)}, y^{(i)})$. Take logs and substitute the factorisation:

$$\begin{aligned} -\log L(\phi) &= -\sum_{i=1}^{N} \left[\log \pi_{y^{(i)}} + \log p_\theta(x^{(i)} \mid y^{(i)}) \right] \\ &= \underbrace{-\sum_{c=1}^{C} N_c \log \pi_c}_{\text{prior part}} \; \underbrace{-\sum_{c=1}^{C} \sum_{i:\, y^{(i)} = c} \log p_\theta(x^{(i)} \mid y = c)}_{\text{one term per class}}, \end{aligned}$$

grouping the prior terms by class (label c occurs N_c times) and the class-conditional terms likewise. The first group depends only on π, the second only on θ, and within it class c's parameters appear only in class c's sum. **Consequence:** the prior is fitted from the counts alone, and *each class model is fitted from that class's examples alone*; nothing is estimated jointly.

Prior. Write $q_c = N_c/N$. Then $\sum_c N_c \log \pi_c = N \sum_c q_c \log \pi_c = -N \, H(q, \pi)$, minus N times the cross-entropy between the empirical class distribution and the model prior. By Gibbs' inequality

$H(q, \pi) \geq H(q)$ with equality iff $\pi = q$ [on your sheet], so the prior part is maximised at $\hat{\pi}_c = N_c/N$ [on your sheet]. Generative training asks each example to be explained *twice*: its class through π and its features through $p_\theta(x \mid y)$.

Two kinds of error of a fitted generative posterior

Shape approximation: the family $p_\theta(x \mid y)$ may not contain the truth; this error does *not* vanish with more data. *Finite-sample estimation*: within the family, $\hat{\phi}$ differs from the best parameter because N is finite; this error shrinks with N.

Purpose. Separate what more data can fix from what only a better model can fix.

Impact. The learning-curve experiment of Section 6 is built on exactly this split.

Confusing the prior with the posterior, or counting the prior per feature

"Attacks are 1%, so this is benign" uses the prior as if it were the posterior; the likelihood ratio can overturn it by a factor of hundreds. In hand computations with several features the prior is counted *once* in the score, never once per feature (a common slip in Naive Bayes). Fix: write the score as $\log \pi_c + \log p(x \mid y = c)$ and only then add per-feature terms.

Key insight. Posterior odds equal prior odds times likelihood ratio: the prior is one factor, the evidence the other, and the generative route makes both explicit.

How the professor asks it

Q. *Show that under the generative model the prior and the class-conditional parameters can be estimated separately, and state what each is estimated from.*
A. Claim: the negative log-likelihood is a sum of a term depending only on π and, for each class c, a term depending only on class c's parameters. Mechanism: $\log p_\phi(x, y) = \log \pi_y + \log p_\theta(x \mid y)$, so summing over i and grouping by label gives $-\sum_c N_c \log \pi_c - \sum_c \sum_{i:y^{(i)}=c} \log p_\theta(x^{(i)} \mid y = c)$. Minimising a sum of terms with disjoint variables is minimising each term. So $\hat{\pi}$ comes from the class counts (and equals N_c/N by Gibbs), and the model of class c from the examples of class c only. Consequence: adding examples of one class never changes another class's fitted model, only the priors.

Q. *Why is the MLE of the prior exactly the class frequency? Give the one-line argument.*
A. Because $\sum_c N_c \log \pi_c = -NH(q, \pi)$ with $q_c = N_c/N$, and the cross-entropy $H(q, \pi) = H(q) + \mathrm{KL}(q \| \pi)$ [on your sheet] is minimised, by Gibbs' inequality, exactly when $\pi = q$. So maximising the prior part means matching the model prior to the observed class proportions.

4 Gaussian class models: LDA and QDA

Multivariate Gaussian class model

$p_\theta(x \mid y = c) = \mathcal{N}(x; \mu_c, \Sigma_c) = (2\pi)^{-d/2} |\Sigma_c|^{-1/2} \exp[-\frac{1}{2}(x - \mu_c)^\top \Sigma_c^{-1} (x - \mu_c)]$ [on your sheet]. μ_c is the centre of class c's cloud; Σ_c its shape: a multiple of I gives circular contours, unequal diagonal entries stretch into an axis-aligned ellipse, off-diagonal entries rotate it (correlated features). The quadratic form is the **squared Mahalanobis distance**, "how many standard deviations from the centre in the direction of x". The prefactor with $|\Sigma_c|$ makes the density integrate to one: a diffuse class spreads its probability thin.

Purpose. The simplest "world" for continuous features.

Impact. The covariance assumption you make becomes the *shape of the boundary* you get.

Closed-form maximum likelihood, one dimension in full

By the split of the likelihood, fix one class and drop c; its $n = N_c$ examples are $x^{(1)}, \dots, x^{(n)}$. The negative log-likelihood is

$$J(\mu, \sigma^2) = \frac{n}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^{n} (x^{(i)} - \mu)^2.$$

Step 1 (mean). For fixed σ^2, $\partial J/\partial\mu = -\frac{1}{\sigma^2} \sum_i (x^{(i)} - \mu) = 0$ gives $\mu = \frac{1}{n} \sum_i x^{(i)}$; J is a convex quadratic in μ, so this is the minimum. **Step 2 (variance).** With $\mu = \hat{\mu}$ and $S = \sum_i (x^{(i)} - \hat{\mu})^2$, put $v = \sigma^2$: $\partial J/\partial v = \frac{n}{2v} - \frac{S}{2v^2} = 0$ gives $v = S/n$; the derivative is negative below and positive above, so it is the minimum. In d dimensions the same two steps give the class mean and the average outer product of the centred examples:

$$\hat{\pi}_c = \frac{N_c}{N}, \qquad \hat{\mu}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} x^{(i)}, \qquad \widehat{\Sigma}_c = \frac{1}{N_c} \sum_{i:\, y^{(i)}=c} (x^{(i)} - \hat{\mu}_c)(x^{(i)} - \hat{\mu}_c)^\top \quad \text{[on your sheet]}.$$

N_c versus $N_c - 1$. N_c is the maximum-likelihood denominator; $N_c - 1$ gives the unbiased sample covariance. Immaterial for large N_c, but they are different estimators: say which one you use.

One dimension by hand: posterior and boundary

Class 0: $\{0, 1, 2, 0, 1, 2\}$; class 1: $\{3, 4, 5\}$. Counts: $\hat{\pi}_0 = 6/9 = 2/3$, $\hat{\pi}_1 = 1/3$. Means: $\hat{\mu}_0 = 6/6 = 1$, $\hat{\mu}_1 = 12/3 = 4$. Variances (divide by N_c): $\hat{\sigma}_0^2 = (1 + 0 + 1 + 1 + 0 + 1)/6 = 2/3$, $\hat{\sigma}_1^2 = (1 + 0 + 1)/3 = 2/3$.

Posterior at $x = 2$. Equal variances, so the prefactors $1/\sqrt{2\pi \cdot 2/3}$ are common and drop. Class 0: $\exp[-(2-1)^2/(2 \cdot \frac{2}{3})] = e^{-0.75} = 0.472$; class 1: $\exp[-(2-4)^2/(2 \cdot \frac{2}{3})] = e^{-3} = 0.050$. Prior-weighted: $\frac{2}{3}(0.472) = 0.315$, $\frac{1}{3}(0.050) = 0.017$. Normalise: $p(y = 0 \mid x = 2) = 0.315/0.332 = 0.950$. Both the distance (one unit from class 0's centre, two from class 1's) and the larger prior push toward class 0.

Boundary. With equal variances the log prior-weighted likelihoods are $\log \hat{\pi}_c - (x - \hat{\mu}_c)^2/(2 \cdot \frac{2}{3})$. Set them equal: $\frac{3}{4}[(x-4)^2 - (x-1)^2] = \log \frac{1}{3} - \log \frac{2}{3} = -\log 2$, i.e. $\frac{3}{4}(15 - 6x) = -0.693$, so $x = (15 + 0.924)/6 = 2.65$. With equal priors the boundary would be the midpoint 2.5; the larger prior of class 0 pushes it 0.15 toward class 1. This is LDA in one dimension: a single threshold.

Generative class score; LDA; QDA

Define $s_{\hat{\phi},c}(x) = \log \hat{\pi}_c + \log p_{\hat{\theta}}(x \mid y = c)$. By Bayes' rule, $\log p_{\hat{\phi}}(y = c \mid x) = s_{\hat{\phi},c}(x) - \log \sum_k \exp[s_{\hat{\phi},k}(x)]$: the fitted posterior is the *softmax of the generative scores* [on your sheet], and prediction uses the scores alone.

- **LDA** (Linear Discriminant Analysis): every class shares one covariance $\widehat{\Sigma}$. Scores are affine in x: $s_c(x) = x^\top \widehat{\Sigma}^{-1} \hat{\mu}_c - \frac{1}{2} \hat{\mu}_c^\top \widehat{\Sigma}^{-1} \hat{\mu}_c + \log \hat{\pi}_c$ [on your sheet]. Boundaries are hyperplanes.
- **QDA** (Quadratic Discriminant Analysis): each class has its own $\widehat{\Sigma}_c$. Scores are quadratic: $s_c(x) = -\frac{1}{2}(x - \hat{\mu}_c)^\top \widehat{\Sigma}_c^{-1} (x - \hat{\mu}_c) - \frac{1}{2} \log |\widehat{\Sigma}_c| + \log \hat{\pi}_c$ [on your sheet]. Boundaries are conic sections.

Purpose. Trade the number of estimated numbers against the flexibility of the boundary.

Impact. LDA estimates $d(d+1)/2$ covariance entries once; QDA estimates them C times. The word *linear* in LDA describes the induced boundary, not the Gaussian density.

Why a shared covariance cancels the quadratic term

Take the log of the Gaussian density: $\log p(x \mid y = c) = -\frac{d}{2}\log(2\pi) - \frac{1}{2}\log|\widehat{\Sigma}| - \frac{1}{2}(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c)$. With a shared covariance the first two terms are the same for every class. Expand the quadratic form, using symmetry of $\widehat{\Sigma}^{-1}$ to merge the two cross terms:

$$(x-\hat{\mu}_c)^\top \widehat{\Sigma}^{-1}(x-\hat{\mu}_c) = x^\top \widehat{\Sigma}^{-1}x - 2x^\top \widehat{\Sigma}^{-1}\hat{\mu}_c + \hat{\mu}_c^\top \widehat{\Sigma}^{-1}\hat{\mu}_c.$$

The only term quadratic in x, $x^\top \widehat{\Sigma}^{-1}x$, does not depend on c and drops out of every comparison between classes. What remains, times $-\frac{1}{2}$, plus $\log \hat{\pi}_c$, is the LDA score, affine in x; two affine functions are equal on a hyperplane. For two classes the boundary is $w^\top x + b = 0$ with $w = \Sigma^{-1}(\mu_1 - \mu_0)$ and $b = -\frac{1}{2}(\mu_1^\top \Sigma^{-1}\mu_1 - \mu_0^\top \Sigma^{-1}\mu_0) + \log\frac{\pi_1}{\pi_0}$ [on your sheet]. With separate covariances, $\log|\widehat{\Sigma}_c|$ and the whole quadratic form depend on c; equality of two scores is a quadratic equation in x (QDA).

Shared versus separate covariance: watch the boundary bend

Setup. Two-dimensional flow features (log duration, log bytes), two classes. Fit LDA and QDA on the same training rows.

Held fixed. Training rows, features, priors (the empirical ones).

What we manipulate. The covariance assumption only: one shared $\widehat{\Sigma}$ (LDA) versus two class-specific $\widehat{\Sigma}_c$ (QDA). Second run: shrink N_c of the attack class from 500 to 50 to 5.

What we measure. The fitted boundary drawn on the plane; validation error; whether $\widehat{\Sigma}_c$ is invertible.

What the plot shows. LDA: a straight line between the two class centres, tilted by $\widehat{\Sigma}^{-1}$. QDA: an ellipse, parabola or hyperbola; when one class is much more diffuse, the tighter class gets a closed region surrounded by the diffuse one. As N_c falls, QDA's boundary for the small class swings wildly from sample to sample; at $N_c = 5 < d+1$ its covariance is singular and no score can be computed.

How to read it. The curvature of the boundary is the visible fingerprint of the assumption. If LDA and QDA give similar validation error, the extra $d(d+1)/2$ parameters per class bought nothing and LDA is the safer choice. If QDA is better and stable, the clouds really do have different shapes.

Parameter count, singular covariance, shrinkage

A symmetric $d \times d$ matrix has $d + d(d-1)/2 = d(d+1)/2$ distinct entries (diagonal plus strictly upper triangle) [on your sheet]: 5050 for $d = 100$, per class under QDA. If a class has fewer than $d+1$ examples its covariance estimate has rank at most $N_c - 1$: singular, not invertible, the score cannot be computed; with slightly more it is unstable. **Shrinkage**: $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma} + \lambda\alpha I$, $0 \le \lambda \le 1$, $\alpha > 0$ a reference variance (e.g. the average diagonal entry) [not on your sheet: know it]. $\lambda = 0$ keeps the empirical matrix; $\lambda = 1$ removes all correlations and gives every direction the same variance; for any $\lambda > 0$ the result is positive definite (positive semidefinite plus a positive multiple of I), hence invertible. λ is chosen on validation.

Purpose. Make the Gaussian route usable when N_c is small relative to d.

Impact. Blends the noisy data-specific matrix with a stable featureless one; pushed to $\lambda \to 1$ and then dropping correlations altogether leads to Naive Bayes.

Shrinkage sweep

Setup. QDA on $d = 20$ flow features, attack class with $N_c = 30$.

Held fixed. The data, the means, the priors.

What we manipulate. $\lambda \in \{0, 0.01, 0.1, 0.3, 0.6, 1\}$ in $\widehat{\Sigma}_\lambda$.

What we measure. Condition number of $\widehat{\Sigma}_\lambda$; training and validation log-loss.

What the plot shows. At $\lambda = 0$ the matrix is near-singular (huge condition number) and validation loss is terrible while training loss is excellent (the class fits its own 30 points). Validation loss falls, reaches a minimum at a moderate λ, then rises slowly toward the spherical model at $\lambda = 1$.

How to read it. The usual U-curve of Chapter 3, with λ as the capacity knob: it shrinks the *effective* number of covariance parameters without changing the model family. Pick the minimum on validation, never on training.

Credit scoring with few defaults

A bank has 40 features and 120 defaulted loans among 20,000. QDA needs $40 \cdot 41/2 = 820$ covariance numbers for the default class from 120 examples: unstable or singular. LDA pools the two classes for one covariance estimated from all 20,000 rows, at the price of assuming the default cloud has the same shape as the non-default cloud. Shrunk QDA sits in between. The usual outcome: LDA or shrunk QDA beats raw QDA on validation, and the "more flexible" model loses because it had to estimate what it could not estimate.

Choosing QDA because it is "more flexible"

Flexibility is only useful if it can be estimated. QDA multiplies the covariance parameters by C; with few examples per class the estimate is noisy (large variance in the Chapter-3 sense) or singular. It inflates training performance and deflates validation performance. Fix: compare LDA, shrunk QDA and QDA on validation; count parameters against N_c before fitting.

Logistic regression versus LDA versus QDA versus Gaussian Naive Bayes

	Logistic regression	LDA	QDA	Gaussian NB
Route	discriminative	generative	generative	generative
Modelled quantity	$p(y \mid x)$, log-odds linear	π_c, $\mathcal{N}(\mu_c, \Sigma)$ shared Σ	π_c, $\mathcal{N}(\mu_c, \Sigma_c)$	π_c, independent 1-D Gaussians per feature
Boundary	hyperplane	hyperplane	quadratic	quadratic (axis-aligned, a diagonal QDA)
Parameters	$(C-1)(d+1)$	$(C-1) + Cd + \frac{d(d+1)}{2}$	$(C-1) + Cd + C\frac{d(d+1)}{2}$	$(C-1) + 2Cd$
Data needed	more	less	most of the three generative	least
Fails when	few labels in high d; needs iterative fitting	class shapes differ; non-Gaussian clouds	$N_c < d+1$ (singular); non-Gaussian	correlated features double-count evidence

Same linear boundary for logistic regression and LDA, obtained by different commitments: LDA assumes Gaussian classes with shared covariance and gets $w = \Sigma^{-1}(\mu_1 - \mu_0)$ in closed form; logistic regression assumes only "log-odds linear" and fits w by iterative maximum likelihood.

Key insight. LDA versus QDA is a decision about how many numbers to estimate; the geometry of the boundary is the visible fingerprint of the assumption you made about the class clouds.

How the professor asks it

Q. *Two classes with equal priors, $\mu_0 = 0$, $\mu_1 = 4$, shared $\sigma^2 = 1$. Where is the boundary? Now set $\pi_0 = 0.9$, $\pi_1 = 0.1$ and recompute. Explain the direction of the move.*
A. Equal priors: the midpoint $x = 2$ (equal variances, equal priors, the log scores differ only in the squared distances). With priors 0.9/0.1: solve $-\frac{1}{2}x^2 + \log 0.9 = -\frac{1}{2}(x-4)^2 + \log 0.1$, i.e. $-\frac{1}{2}x^2 + \frac{1}{2}(x-4)^2 = \log 0.1 - \log 0.9 = -\log 9$; the left side is $-4x + 8$, so $4x - 8 = \log 9 = 2.197$ and $x = 2.55$. The boundary moves *away* from the frequent class: a point must be further toward class 1's centre before the evidence overcomes the 9:1 prior.

Q. *With $d = 10$ features and $C = 3$ classes, how many covariance parameters do QDA and LDA estimate? A class has $N_c = 5$ examples: what goes wrong and what are the two remedies?*
A. A 10×10 symmetric matrix has $10 \cdot 11/2 = 55$ entries. QDA: $3 \times 55 = 165$; LDA: 55. With $N_c = 5$ the centred examples span at most 4 directions, so $\widehat{\Sigma}_c$ has rank $\leq 4 < 10$: singular, the score needs $\widehat{\Sigma}_c^{-1}$ and cannot be computed. Remedies: shrink, $\widehat{\Sigma}_\lambda = (1-\lambda)\widehat{\Sigma}_c + \lambda\alpha I$, invertible for any $\lambda > 0$; or impose structure, shared covariance (LDA) or a diagonal one (Gaussian Naive Bayes, $2d = 20$ numbers per class).

Q. *Derive why LDA's boundary is linear although the Gaussian density is not, and name the single term whose cancellation is responsible.*
A. The log density is a constant minus $\frac{1}{2}\log|\Sigma|$ minus $\frac{1}{2}(x-\mu_c)^\top\Sigma^{-1}(x-\mu_c)$. Expanding the quadratic form gives $x^\top\Sigma^{-1}x - 2x^\top\Sigma^{-1}\mu_c + \mu_c^\top\Sigma^{-1}\mu_c$. The term $x^\top\Sigma^{-1}x$ is the only one quadratic in x, and with a *shared* Σ it is identical for every class, as is $\log|\Sigma|$; both cancel when two class scores are compared. What remains is affine in x, so the set where two scores tie is a hyperplane. If the covariances differ, neither term cancels and the tie set is quadratic (QDA).

5 Naive Bayes: independence given the class

Naive Bayes

Assume the features are **conditionally independent given the class**: $p_\theta(x \mid y = c) = \prod_{j=1}^{d} p_\theta(x_j \mid y = c)$ [on your sheet]. Taking logs the score becomes *additive*, $s_c(x) = \log\hat{\pi}_c + \sum_j \log p_{\hat{\theta}}(x_j \mid y = c)$ [on your sheet]: a ledger that starts with the prior evidence and adds one line per feature, each from a *one-dimensional* model of that feature within class c.

Purpose. Replace one hopeless d-dimensional density estimate by d easy one-dimensional ones.

Impact. Stable with little data; each feature may get its own family (Gaussian for a measurement, Gamma or log-normal for a skewed positive quantity, Poisson for a count, histogram for an irregular one). Two facts make it far more useful than the assumption suggests: a *misspecified density can still rank classes correctly* (getting the largest score right is weaker than getting every density right), and the one-dimensional fits are cheap. The main risk: correlated features *repeat the same evidence*; the classifier may still rank well while its probabilities become too extreme.

The duplicated-feature experiment: ranking survives, calibration dies

Setup. Gaussian Naive Bayes on flow data with features (duration, packets, bytes).

Held fixed. Training and validation rows; the prior; the decision threshold 0.5.

What we manipulate. Add a copy of the "bytes" column (or a feature that is 0.98-correlated with it, such as bytes per packet times packets) once, twice, five times.

What we measure. Validation accuracy; a reliability diagram (among flows given probability

u, the observed fraction of attacks); the average of $\max_c \hat{p}(y = c \mid x)$.

What the plot shows. Accuracy barely moves: the ranking is driven by the same information. The reliability curve bends into an S: predictions of 0.7 turn into 0.9, then 0.99, with the observed fraction unchanged. The mean top probability climbs toward 1.

How to read it. Each copy adds the same log-likelihood term again, so the evidence is counted k times and the log-odds are multiplied by k. The model becomes overconfident, which matters for cost thresholds, soft voting (Chapter 5) and any probability-based policy. Fix: remove redundant features or use a model that sees correlations (LDA, logistic regression).

Gaussian Naive Bayes and variance smoothing

For numerical features, $x_j \mid (y = c) \sim \mathcal{N}(\hat{\mu}_{cj}, \hat{\sigma}^2_{cj})$ with the one-dimensional MLEs feature by feature. Two problems: a fitted variance of 0 (all class-c values of feature j coincide) makes the density undefined; a merely tiny variance lets one feature's narrow density dominate the whole score. **Variance smoothing**: $\tilde{\sigma}^2_{cj} = \hat{\sigma}^2_{cj} + \lambda s^2_j$ with $\lambda > 0$ and s^2_j a reference scale such as the overall training variance of feature j [not on your sheet: know it]. Small λ prevents instability; if its value changes predictions, select it on validation.

Purpose. Keep one feature from vetoing the class.

Impact. Example: class c has $\hat{\sigma}^2 = 1$ for feature 1 and 10^{-6} for feature 2 (every training example of class c had the same port number). A test point whose feature 2 differs by 0.01 contributes $-\frac{1}{2}(0.01)^2/10^{-6} = -50$ to the score, dwarfing anything feature 1 can say. With $\lambda s^2_2 = 0.01$ added, the contribution is $-\frac{1}{2}(10^{-4})/0.01 \approx -0.005$: feature 2 still counts, but no longer vetoes.

Naive Bayes variants, by feature type

Variant	Feature model	Typical use
Gaussian	continuous Gaussian per feature	measurements, flow statistics
Bernoulli	binary presence/absence	indicators (has link, from known sender)
Multinomial	nonnegative event or token counts	documents, event counts
Hybrid	distribution chosen per feature type	mixed tabular data

A hybrid model multiplies contributions from different feature-specific distributions under *one* class prior; the prior appears once in the score, never once per feature.

Multinomial Naive Bayes for text

Fix a vocabulary of V words; a document is its count vector $x = (x_1, \ldots, x_V)$. For class c let $\theta_{cj} = p(\text{token} = j \mid y = c)$, $\sum_j \theta_{cj} = 1$. Ignoring the multinomial coefficient (same for every class, cancels in Bayes' rule), $p_\theta(x \mid y = c) \propto \prod_{j=1}^{V} \theta_{cj}^{x_j}$ [on your sheet] with $0^0 = 1$. Class c is a bag of words with proportions $\theta_{c\cdot}$; a document is produced by drawing words from the bag. "free free prize" with vocabulary (free, win, prize, meeting, project, report) has $x = (2, 0, 1, 0, 0, 0)$ and likelihood $\propto \theta^2_{c,\text{free}}\theta_{c,\text{prize}}$: the same word twice is twice the evidence; absent words contribute the factor 1; word order is ignored.

Purpose. The spam-filter model.

Impact. Everything is a count; estimation is counting; the only subtlety is a count of zero.

Zero counts, additive smoothing, and the "free meeting" numbers

Training counts, seven tokens per class:

	free	win	prize	meeting	project	report
spam	3	2	2	0	0	0
ham	0	0	0	3	2	2

Without smoothing $\hat{\theta}_{cj} = n_{cj}/7$ and "free meeting" gives $p(x \mid \text{spam}) \propto \frac{3}{7} \cdot \frac{0}{7} = 0$, $p(x \mid \text{ham}) \propto \frac{0}{7} \cdot \frac{3}{7} = 0$: one unseen word zeroes an entire product; here both vanish and the posterior is 0/0. In general a single word never seen in a class vetoes that class regardless of the other evidence.
Additive smoothing [on your sheet]:

$$\hat{\theta}_{cj} = \frac{n_{cj} + \alpha}{\sum_{\ell} n_{c\ell} + \alpha V}, \qquad p(x_j = v \mid y = c) = \frac{n_{cjv} + \alpha}{n_{cj} + \alpha K_j} \text{ for a categorical feature with } K_j \text{ levels.}$$

Properties, each one line: numerators are $\geq \alpha > 0$ and summing $n_{cj} + \alpha$ over j gives exactly the denominator, so the estimates are positive and sum to one; as $\alpha \to 0$ they tend to the raw frequencies; dividing numerator and denominator by α, as $\alpha \to \infty$ they tend to $1/V$ (uniform). The estimate is what you get by adding α imaginary occurrences of every word to every class.
With $\alpha = 1$**,** $V = 6$ (Laplace): spam counts $(4, 3, 3, 1, 1, 1)$ over $7 + 6 = 13$, ham $(1, 1, 1, 4, 3, 3)$ over 13. Equal priors $\frac{1}{2}$:

- "free meeting": spam $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{1}{13} = \frac{2}{169}$, ham $\frac{1}{2} \cdot \frac{1}{13} \cdot \frac{4}{13} = \frac{2}{169}$; posterior $\frac{1}{2}$, the evidence balances exactly.
- "free prize": spam $\frac{1}{2} \cdot \frac{4}{13} \cdot \frac{3}{13} = \frac{12}{338}$, ham $\frac{1}{2} \cdot \frac{1}{13} \cdot \frac{1}{13} = \frac{1}{338}$; $p(\text{spam} \mid x) = 12/13 = 0.92$.
- "free free prize": spam $\frac{1}{2}(\frac{4}{13})^2 \frac{3}{13} = \frac{48}{4394}$, ham $\frac{1}{2}(\frac{1}{13})^3 = \frac{1}{4394}$; $p(\text{spam} \mid x) = 48/49 = 0.98$. The repeated word counts twice, as the multinomial says it should.

The α sweep

Setup. Multinomial Naive Bayes on 2,000 labelled emails, vocabulary $V = 10{,}000$.

Held fixed. Training and validation emails, the vocabulary, the priors.

What we manipulate. $\alpha \in \{0, 10^{-3}, 10^{-2}, 0.1, 1, 10, 100\}$.

What we measure. Validation log-loss and accuracy; the fraction of validation emails containing at least one word unseen in one class.

What the plot shows. At $\alpha = 0$ log-loss is infinite (zero probabilities) and accuracy is erratic: every unseen word vetoes a class. Log-loss drops steeply for tiny α, bottoms out around α between 0.1 and 1, then rises as α pulls every $\hat{\theta}_{cj}$ toward $1/V$ and the classes stop being distinguishable (at $\alpha = 100$ with counts $(5, 0, 3, 2)$, $V = 4$, the estimates are $(0.256, 0.244, 0.251, 0.249)$, nearly uniform).

How to read it. α is a capacity knob in disguise: small α trusts the counts (high variance, vetoes), large α trusts the uniform prior (high bias). Choose it on validation. The unseen-word fraction tells you *why* $\alpha = 0$ fails: it is a property of the vocabulary size against the training size, not of the classifier.

Unseen categories

A categorical value that appears at validation or test time but never in training (a new port number) must have a defined representation: create an *other* level during preprocessing, count it among the K_j levels, map every unseen value to it. Smoothing then gives it a small positive probability instead of a crash.

Purpose. Make the pipeline robust to deployment novelty at the feature level.

Impact. Without it, the model cannot even compute a score for a new value.

Spam at day one versus spam at scale

200 labelled emails, 10,000-word vocabulary. Multinomial Naive Bayes estimates $2 \times 10{,}000$ smoothed word probabilities from simple counts and works reasonably from day one. Logistic regression on the same counts must locate a 10,000-dimensional hyperplane from 200 points: badly underdetermined, it overfits. With 2 million labelled emails the picture flips: logistic regression (or a network) learns interactions such as "*free* is spammy unless *invoice* is nearby", which the independence assumption forbids Naive Bayes from ever representing.

The three hand-computation slips

(1) Counting the prior once per feature instead of once per class, which inflates the prior's weight by a factor d. (2) Smoothing after normalising instead of before: add α to every count and αV to the total, then divide. (3) Multiplying raw probabilities for many features and underflowing or mis-multiplying: work in logs when there are more than three or four features, compare unnormalised scores first, normalise only if a posterior is asked for.

Gaussian NB versus LDA, and the hidden QDA

- Gaussian NB with class-specific per-feature variances is a *diagonal QDA*: scores quadratic in x, axis-aligned ellipses, $2d$ numbers per class. So its boundary can curve even though each feature is Gaussian.
- LDA with a diagonal shared covariance is a diagonal LDA: linear boundary, d variances in total.
- Full LDA sees correlations and never double-counts; NB ignores them and may, but needs far less data.

Key insight. Naive Bayes trades one strong assumption for stability; its ranking often survives the assumption being false, its probabilities do not.

How the professor asks it

Q. *Two features (has link, from known sender). Spam rates* $(0.9, 0.1)$*, ham rates* $(0.2, 0.8)$*, priors* 0.5 *each. Classify an email that has a link and comes from a known sender, and give the posterior. Then explain in one sentence what Bernoulli Naive Bayes assumed to let you multiply.*
A. Ham, with posterior 0.64. Spam: $0.5 \times 0.9 \times 0.1 = 0.045$; ham: $0.5 \times 0.2 \times 0.8 = 0.08$; normalise, $0.08/0.125 = 0.64$. The multiplication assumes that, within each class, having a link and coming from a known sender are independent events, so the class-conditional probability of the pair is the product of the two marginals.

Q. *Vocabulary* $V = 4$*, class counts* $(5, 0, 3, 2)$*. Give the smoothed probabilities for* $\alpha = 1$ *and say what happens as* $\alpha \to \infty$ *and why that is a bias, not an error.*
A. $\alpha = 1$: $(6, 1, 4, 3)/14$. As $\alpha \to \infty$ every estimate tends to $1/4$ regardless of the counts ($\alpha = 100$ already gives $(0.256, 0.244, 0.251, 0.249)$): the class models become identical and the classifier returns the prior. It is bias in the Chapter-3 sense: a systematic pull toward uniform that no amount of data in the counts can undo at that α; the correct response is to choose α on validation, where the pull is traded against the zero-count variance.

Q. *Why can a Gaussian Naive Bayes boundary be curved when every feature is Gaussian? Which assumption would make it straight?*
A. Because each class has its own variance per feature, the score $\log \pi_c - \sum_j [(x_j - \mu_{cj})^2/(2\sigma_{cj}^2) + \frac{1}{2}\log \sigma_{cj}^2]$ is quadratic in x with class-dependent coefficients: a diagonal QDA, so the tie set is quadratic. Sharing the variances across classes ($\sigma_{cj}^2 = \sigma_j^2$) makes the x_j^2 terms cancel, as in LDA, and the boundary becomes a hyperplane.

6 Comparing the routes: the data-size experiment

What decides between the routes

Generative structure can *reduce estimation error*: a restricted description of each class has few parameters and is fitted reliably from little data, provided the assumptions are approximately right. A wrong class-conditional family creates *approximation bias* that does not vanish with more data. A flexible discriminative model may reach lower error eventually because it spends no capacity on aspects of X irrelevant to Y.

Purpose. Turn "which is better" into "which assumptions are credible, and what is the deployment objective".

Impact. The expected picture, error versus N: generative lower at small N, discriminative lower at large N. Not a law: the crossing point, and whether it exists, depends on the distribution, the two families, regularisation and optimisation.

Learning curves of Naive Bayes versus logistic regression as N grows

Setup. Spam data, vocabulary V fixed, a fixed test set of 20,000 emails never used for anything else.

Held fixed. The features, the test set, the two model families, the smoothing α and the regularisation λ (each chosen once on validation at a moderate N).

What we manipulate. The number of labelled training emails $N \in \{50, 200, 1{,}000, 10{,}000, 100{,}000, 2\,\text{million}\}$, on a log axis; for each N repeat with 10 random training subsets to see variability.

What we measure. Test error of each model at each N, mean and spread across the 10 subsets.

What the plot shows. Two decreasing curves. Naive Bayes starts low and flattens early at a floor (its approximation bias: the independence assumption cannot represent interactions). Logistic regression starts high, with wide spread (underdetermined), falls steadily and crosses below Naive Bayes somewhere between 10^3 and 10^5, then keeps improving slowly.

How to read it. Where the curves cross is *the* fact to report: left of it, the generative assumptions are worth more than their bias; right of it, data has replaced assumptions. A flat generative curve with a gap to the discriminative one is approximation error (a better class model, not more data, is the fix). Wide spread at small N is estimation error (variance). If the curves never cross, either the assumptions are nearly true or N never got large enough.

Settling the choice by slogan

"Generative is for small data", "discriminative is always better": both are tendencies, not theorems. People choose by the label and skip the validation. What it produces: a Naive Bayes stuck at its bias floor when millions of labels were available, or a 10,000-dimensional logistic regression fitted to 200 rows. Fix: ask the three questions (credible class model? prediction with many labels? raw high-dimensional input?) and then *compare under deployment-like validation*, same split, same rows.

The three model-choice questions

1. *Can a credible class-conditional model $p_\theta(x \mid y)$ be stated?* If yes, its structure may reduce the labelled data required.
2. *Is the goal accurate prediction with many labelled examples?* A flexible discriminative model avoids unnecessary assumptions on the inputs.
3. *Is the raw input complex and high-dimensional?* Learn a representation first, then compare

simple classification heads; that is more credible than modelling the raw density.

Deployment-like validation decides between credible alternatives; the labels "generative" and "discriminative" do not replace empirical comparison.

Key insight. The crossing point of the two learning curves is where data has bought enough to replace assumptions; left of it structure wins, right of it flexibility wins.

How the professor asks it

Q. *Describe an experiment that shows the generative route beating the discriminative one at small N and losing at large N, and say which curve shape corresponds to approximation error and which to estimation error.*
A. Plot test error against the number of labelled training examples on a log axis for Naive Bayes and logistic regression, same features, same fixed test set, repeated over random training subsets. Generative: lower at small N, flattening at a floor; discriminative: higher and widely spread at small N, falling and crossing below. The flat floor that more data does not lower is approximation error (the family cannot express the truth); the spread across subsets that shrinks as N grows is estimation error (variance). The crossing exists only if the generative assumptions are wrong enough to leave a floor above what the discriminative model can reach.

Q. *A colleague says a discriminative model "cannot be worse than a generative one on the same features because it makes fewer assumptions". Give the counter-argument.*
A. Fewer assumptions mean more to estimate from the same data: with 200 emails and 10,000 features, logistic regression must locate a hyperplane from far too few points and overfits, while multinomial Naive Bayes estimates $2V$ smoothed frequencies from counts and works. Assumptions are a form of prior information; when they are approximately true they lower variance more than they add bias. The claim is only right asymptotically, when data has made the assumptions unnecessary.

7 Calibration, class imbalance and prior shift

Calibration

A probabilistic classifier is **calibrated** if, among cases assigned probability u, approximately a fraction u are positive. Calibration concerns the *interpretation* of probabilities; a decision threshold from costs concerns the *action*. Separate questions.

Purpose. Know whether "0.8" means eight out of ten.

Impact. Every formula that uses the probability as a probability (cost threshold, prior correction, soft voting) silently assumes calibration. Naive Bayes with correlated features is a standard example of a well-ranking, badly-calibrated model.

Class imbalance versus class-prior shift

Imbalance: one class is rare, and training and deployment share the same distribution. Not an error; the posterior *should* reflect the prevalence. The only problem is finite minority data: with few positives the minority distribution or the boundary is poorly estimated, and weighting or resampling may help estimation. Otherwise its benefit is limited and the probabilities may need recalibration. **Prior shift**: between training (t) and deployment (d) the class-conditionals are unchanged but the priors differ, $p_d(x \mid y = c) = p_t(x \mid y = c)$, $\pi_{d,c} \neq \pi_{t,c}$ [not on your sheet: know it]. Example: attacks are 20% of the training data but 5% of deployment traffic.

Purpose. Two situations constantly confused; one needs no correction, the other needs one even

with abundant data.

Impact. Imbalance is an estimation issue; shift is a modelling mismatch that the formulas below repair exactly.

Generative correction: swap the priors

At deployment Bayes' rule uses π_d and $p_d(x \mid y)$; under prior shift the latter equals $p_t(x \mid y)$, which is what the fitted $p_{\hat{\theta}}(x \mid y)$ estimates. Substituting,

$$p_d(y = c \mid x) = \frac{\pi_{d,c}\, p_{\hat{\theta}}(x \mid y = c)}{\sum_k \pi_{d,k}\, p_{\hat{\theta}}(x \mid y = k)} \quad \text{[not on your sheet: know it]}.$$

No class-conditional model is refitted: the prior is an explicit, separately estimated component of the generative model, so you replace it.

Discriminative correction: reweight the odds

Let $r_t(x) = p_t(y = 1 \mid x)$, $r_d(x) = p_d(y = 1 \mid x)$, $\pi_t = p_t(y = 1)$, $\pi_d = p_d(y = 1)$. **Step 1 (odds form).** At training, $\dfrac{r_t}{1 - r_t} = \dfrac{\pi_t\, p(x \mid y = 1)}{(1 - \pi_t)\, p(x \mid y = 0)}$ (divide the two Bayes numerators; the denominator cancels). Likewise at deployment with π_d. **Step 2 (shared likelihood ratio).** Under prior shift $p(x \mid y = 1)/p(x \mid y = 0)$ is the same at both times. Solve the training odds for it and substitute into the deployment odds:

$$\frac{r_d}{1 - r_d} = \frac{\pi_d/(1 - \pi_d)}{\pi_t/(1 - \pi_t)} \cdot \frac{r_t}{1 - r_t}.$$

Step 3 (back to a probability). If odds $= A/B$ then probability $= A/(A+B)$. With $A = \frac{\pi_d}{\pi_t} r_t$ and $B = \frac{1-\pi_d}{1-\pi_t}(1 - r_t)$,

$$r_d(x) = \frac{(\pi_d/\pi_t)\, r_t(x)}{(\pi_d/\pi_t)\, r_t(x) + \frac{1-\pi_d}{1-\pi_t}\,[1 - r_t(x)]} \quad \text{[not on your sheet: know it]}.$$

Reading. Multiply the training probability of class 1 by "how much rarer class 1 became", the probability of class 0 by "how much commoner class 0 became", renormalise. Attacks 20% → 5%: $\pi_d/\pi_t = 0.25$, $(1 - \pi_d)/(1 - \pi_t) = 0.95/0.80 = 1.1875$. A flow with $r_t = 0.5$: $A = 0.125$, $B = 0.59375$, $r_d = 0.125/0.71875 = 0.17$. A training coin flip is a 17% attack probability in deployment. Other values: $r_t = 0.2 \to 0.05$, $r_t = 0.8 \to 0.46$, $r_t = 0.9 \to 0.65$. The correction assumes r_t is a *calibrated* training posterior.

The prior-shift experiment: train at 20%, deploy at 5%

Setup. Build a training set with 20% attacks and a deployment-like test set with 5% attacks by subsampling attacks from the same pool (so the class-conditionals are identical by construction). Fit Gaussian Naive Bayes and logistic regression on the training set.

Held fixed. The fitted class-conditionals and the fitted logistic model; the test rows; the cost threshold.

What we manipulate. Whether the correction is applied: (a) none; (b) generative: replace $\hat{\pi} = (0.8, 0.2)$ by $(0.95, 0.05)$; (c) discriminative: reweight the odds with $\pi_d/\pi_t = 0.25$ and 1.1875.

What we measure. On the test set: log-loss, reliability diagram, false-positive rate and recall at the cost threshold, number of alarms.

What the plot shows. Uncorrected models are overconfident about attacks (reliability curve below the diagonal: predicted 0.5, observed 0.17) and raise about four times too many alarms.

After correction the reliability curve returns to the diagonal, log-loss falls, alarms drop to the level the costs intended, recall falls slightly.

How to read it. Accuracy may barely move, which is why people miss the problem: the *ranking* is unchanged by a monotone odds reweighting; what changes is every decision that reads the probability as a probability. Also note that the correction is exact for the generative model and only as good as the calibration of r_t for the discriminative one.

Fraud after a marketing campaign

A fraud model was trained on a month where 2% of transactions were fraudulent. A campaign triples legitimate volume; fraud stays constant, so the deployment prior is about 0.7%. The transaction patterns of fraud and of legitimate purchases did not change: pure prior shift. Without correction, the analysts see the same number of alarms per fraud but three times as many alarms per day and a precision that collapsed. Swapping the prior (generative) or reweighting the odds (discriminative) restores the intended precision; retraining is unnecessary and would waste the month's labels.

Resampling the training set and then trusting the probabilities

Oversampling the minority or undersampling the majority *changes the effective prior*: a model trained on a balanced resample outputs posteriors for $\pi = 0.5$, not for the real prevalence. The probabilities are inflated for the rare class by the odds factor $\frac{0.5/0.5}{\pi/(1-\pi)}$; alarm counts and precision are wrong. Tempting because balanced data "looks fair" and accuracy on the resample looks good. Fix: either keep the natural prior with enough minority data, or resample and correct back with the odds formula (π_t = resampled prior, π_d = true prior); and always apply the prior correction *before* the cost threshold, never after.

Imbalance versus shift; calibration versus threshold

	Imbalance	Prior shift
What is true	same distribution at training and deployment, one class rare	class-conditionals equal, priors differ
What is manipulated	the number of minority examples (weights, resampling)	the prior, at prediction time
Needs a correction?	no, with enough minority data; weighting helps only estimation	yes, always, even with abundant data
What goes wrong otherwise	noisy boundary, poor minority recall	systematically miscalibrated probabilities, wrong alarm volume

Calibration is about whether 0.8 means eight in ten (a property of the model); the **threshold** is about which action to take at 0.8 (a property of the costs). Correct the prior first, then threshold.

Key insight. Imbalance is an estimation problem and shift is a modelling mismatch; the generative route corrects shift by replacing one component, the discriminative route only by reweighting odds it must first have calibrated.

How the professor asks it

Q. *A detector was trained with* 20% *attacks; at deployment* 5% *of flows are attacks and attack behaviour is unchanged. A flow receives training posterior* 0.5. *Derive the deployment posterior and interpret the result. What must be true of the training posterior for this to be exact?*
A. Deployment posterior 0.17. Odds reweighting: posterior odds equal prior odds times the likelihood ratio, and the likelihood ratio is the same at both times under prior shift, so the

deployment odds are the training odds times $\frac{\pi_d/(1-\pi_d)}{\pi_t/(1-\pi_t)} = \frac{0.05/0.95}{0.20/0.80} = 0.2105$. Training odds 1; deployment odds 0.2105; probability $0.2105/1.2105 = 0.17$. Equivalently $A = 0.25 \times 0.5 = 0.125$, $B = 1.1875 \times 0.5 = 0.594$, $A/(A+B) = 0.17$. Interpretation: a training coin flip is an unlikely attack in deployment because attacks are four times rarer there. Exactness requires r_t to be calibrated: the formula transforms a true training posterior, not an arbitrary score.

Q. *Explain why oversampling the attack class to* 50% *and then using the model's probabilities with a cost threshold gives the wrong number of alarms, and how to fix it without retraining.*
A. The resampled training set has prior 0.5, so the fitted posterior is $p(y = 1 \mid x)$ under $\pi_t = 0.5$; at deployment the prior is the true π_d, so every attack probability is inflated by the odds factor $\frac{\pi_d/(1-\pi_d)}{1}$ being ignored. The cost threshold then fires far too often. Fix: apply the discriminative correction with $\pi_t = 0.5$ and π_d the true prevalence (multiply class-1 probability by $\pi_d/0.5$, class-0 by $(1-\pi_d)/0.5$, renormalise), then threshold. No refit is needed; the ranking was never the problem.

Q. *Give one situation where class imbalance should be left alone and one where it should be addressed, and say what "addressed" changes.*
A. Leave alone: 1% attacks but 50,000 labelled attack flows, same distribution at deployment; the posterior correctly reflects the prevalence and the boundary is well estimated. Address: 1% attacks with 40 labelled attack flows; the minority distribution is too poorly estimated to locate the boundary, so weighting or resampling reduces estimation variance. "Addressed" changes the effective prior and therefore the calibration, which must then be corrected back; it does not change what the true posterior is.

8 Unfamiliar inputs, combined systems, language models

Reject option via a compatibility check

Choose the most probable known class $c^* = \arg\max_c p_{\hat{\phi}}(y = c \mid x)$, then compare the fitted class likelihood with a class-specific threshold:

$$\hat{y} = \begin{cases} c^* & \text{if } \log p_{\hat{\theta}}(x \mid y = c^*) \geq \tau_{c^*}, \\ \text{reject} & \text{otherwise} \end{cases} \quad \text{[not on your sheet: know it].}$$

Two questions instead of one: *which known class fits best* (the posterior) and *does even the best class fit well enough* (the likelihood against a threshold). Only a generative model can answer the second, because only it has a notion of what inputs of class c look like.

Purpose. Let the system say "I do not recognise this".

Impact. A new attack mechanism far from every known class gets rejected or sent to review instead of a confident wrong label. Caveats: τ_c must be validated on data that reproduce the intended novelty (attack mechanisms excluded from training); a low likelihood is evidence of incompatibility, not a guarantee of detection; high-dimensional likelihoods behave counterintuitively and a misspecified class model can give low likelihood to valid cases.

Held-out-mechanism experiment for the reject threshold

Setup. Flow data with six attack families. Train a generative classifier on five families plus benign.

Held fixed. The trained class models and priors; the posterior-based decision.

What we manipulate. The threshold τ_c on $\log p(x \mid y = c^*)$, swept from very low (never reject) to high (reject most).

What we measure. On validation: rejection rate on known-family flows (false rejects) and on

the sixth, held-out family (true rejects); the known-class accuracy of the non-rejected flows.

What the plot shows. A trade-off curve: as τ rises, true rejects of the novel family climb toward one while false rejects of known flows climb more slowly; accuracy on the accepted flows rises slightly because the uncertain ones are gone.

How to read it. The sixth family is the novelty the deployment will face; a threshold tuned without it would be tuned to the wrong question. Compare with the softmax-only system, whose top probability on the novel family is high and gives no handle at all. The threshold is a hyperparameter: pick it on validation, report on an untouched test family if you have one.

Combined systems

A discriminative model ranks the known classes, $p_{\hat{\theta}_d}(y = c \mid x)$. A generative or anomaly model computes $\log p_{\hat{\theta}_g}(x \mid y = c)$ for the predicted class. The final policy accepts the known class, requests human review, or rejects the input as unfamiliar. The discriminative model is good at the ranking; the generative model is the plausibility check it lacks.

Purpose. Get the best of both routes.

Impact. Three outcomes (accept, review, reject) instead of a forced choice among known classes.

Generative language models used for classification

An autoregressive language model is trained generatively by factorising a token sequence, $p_\theta(w_{1:T}) = \prod_{t=1}^{T} p_\theta(w_t \mid w_{<t})$, with next-token prediction as the pretraining objective. A *sequence-classification head* instead models $p_\psi(y \mid w_{1:T})$ and minimises the discriminative objective $-\sum_i \log p_\psi(y^{(i)} \mid w^{(i)}_{1:T_i})$: one label for the entire sequence, not one per token. Alternatively a prompt asks the model to generate or score label tokens directly.

Purpose. See both routes inside one modern system.

Impact. Generative pretraining learns the representation; a discriminative objective learns the label; the two words describe the direction of each objective, not the architecture.

Lab 2 at a glance

The lab compares the routes on cybersecurity flow data: Gaussian, multinomial and hybrid Naive Bayes for numerical, service and mixed feature sets, against logistic regression and a decision tree; then limited training data (20% of each class), deployment prior shift ($\pi_d = 0.05$, corrected by swapping priors for the generative models and by odds reweighting for the discriminative ones), and multiclass classification with and without destination-port information. Every model uses the same fixed stratified split and the same rows, so that differences reflect the models rather than the data.

Validating the reject threshold on the known classes only

Tempting because that is the data at hand. It produces a threshold that rejects a comfortable fraction of known flows and tells you nothing about novel ones; at deployment the new attack family sails through with a confident label. Fix: hold out an entire mechanism or family for validation of τ, the grouped-split idea of Chapter 3 applied to classes.

Softmax confidence versus likelihood check

- **Top softmax probability**: relative fit among the known classes; normalised to one, so it cannot be low for every class at once; available from any classifier; high on novel inputs.
- **Class likelihood** $\log p(x \mid y = c^*)$: absolute compatibility of the input with the best class; can be low for all classes; needs a generative or anomaly model; thresholded with a

validated τ_c.

- **Use**: softmax to rank, likelihood to decide whether to trust the ranking at all.

Key insight. Softmax answers "which of my classes is best"; the likelihood answers "is any of them any good"; a reject option needs the second question and only a generative model can ask it.

How the professor asks it

Q. *A three-class network outputs* $(0.96, 0.03, 0.01)$ *on an input that belongs to none of its classes. Explain why this is not a bug of the training, and what you would add to the system to catch it.*
A. Not a bug: the cross-entropy objective only ever asked the network to explain the labels given the inputs, so its softmax compares the three known classes and nothing else; 0.96 says "cat explains this far better than dog or horse", which can be true of a toaster. The number measures relative, not absolute, fit. Add a compatibility check: a generative or anomaly model giving $\log p(x \mid y = c^*)$, compared with a threshold validated on held-out novel inputs; reject or send to review when the best class fits badly.

Q. *Why can the reject option (the compatibility check) not be implemented with a discriminative classifier alone?*
A. Because the check needs $p(x \mid y = c)$, a model of what inputs of class c look like, and a discriminative classifier never estimates any distribution over x: it models $p(y \mid x)$ only, and its outputs are normalised across classes, so "all classes fit badly" is invisible to it. Thresholding the top softmax probability is not a substitute, since that probability can be near one on unfamiliar inputs.

Coverage of the textbook chapter

- **Section 1:** symbol table; two routes; decision known, posterior not; definition of discriminative and generative; where each route shows up; "generative" is a direction, not a task.
- **Section 2:** scores, sigmoid, logistic regression; sigmoid properties; theorem on decision boundaries and the cost shift; softmax and its facts; cross-entropy equals conditional maximum likelihood; deep classifiers; what the objective does not provide; confident about nonsense; the imbalance remark; the technique box on scores and probabilities.
- **Section 3:** generative story; Bayes' rule and the rare-attack table; the likelihood splits by class; the MLE prior via Gibbs; shape error versus finite-sample error.
- **Section 4:** Gaussian model; Gaussian MLE with N_c versus $N_c - 1$; the one-dimensional example (posterior 0.950, boundary 2.65); generative score; LDA theorem; QDA; covariance parameter count; shrinkage; the by-hand technique; the classics (boundary at 2 and 2.55; 165 and 55 parameters; singular covariance).
- **Section 5:** Naive Bayes; Gaussian NB and variance smoothing; the variants table; multinomial NB; zero counts; additive smoothing and its properties; "free meeting", "free prize", "free free prize"; unseen categories; the by-hand technique; the classics (smoothing with $\alpha = 1$ and 100, Bernoulli 0.64, curved NB boundary).
- **Section 6:** the comparison table; the interpretation; 200 versus 2 million emails; the three model-choice questions.
- **Section 7:** calibration; imbalance; prior shift; generative correction; discriminative correction with the $0.5 \to 0.17$ numbers; the gotchas (imbalance is not shift, resampling changes the prior, correction before threshold).
- **Section 8:** confidence is not familiarity; the reject option; combined systems; language models; Lab 2.

The ten mistakes of this chapter

1. Reading $\max_c p(y = c \mid x)$ as "this input is normal". Softmax compares known classes only.
2. Using the prior as if it were the posterior ("attacks are 1%, so benign"). Posterior odds are prior odds times the likelihood ratio.
3. Believing costs bend the boundary. They slide it along the score axis; only the score family bends it.
4. Counting the prior once per feature in a Naive Bayes hand computation. Once per class.
5. Smoothing after normalising, or not at all. Add α to every count and αV to the total, then divide; $\alpha = 0$ lets one unseen word veto a class.
6. Picking QDA "because it is more flexible" with N_c close to d. Singular or unstable covariance; count $d(d+1)/2$ per class first; shrink or share.
7. Saying LDA is linear because the Gaussian is linear. It is linear because the shared $x^\top \Sigma^{-1} x$ cancels between classes.
8. Treating imbalance and prior shift as the same problem. Imbalance with enough data needs nothing; shift needs a correction even with abundant data.
9. Resampling to 50/50 and then trusting the probabilities or alarm counts. The effective prior changed; correct the odds back, then threshold.
10. Settling generative versus discriminative by slogan instead of by the learning-curve experiment under deployment-like validation.

Chapter 4 in ten lines

1. The decision rule is known from Chapter 2; the posterior must be learned. Two routes: model $p(y \mid x)$ directly, or model $p(x \mid y)$ and π and apply Bayes' rule.
2. Discriminative: score, sigmoid or softmax, cross-entropy. Log-odds equal the score; the boundary is $s = \log(C_{\mathrm{FP}}/C_{\mathrm{FN}})$; costs translate it, the score family shapes it.
3. Cross-entropy minimisation is conditional maximum likelihood: the labels given the inputs, never the inputs.
4. Generative: joint $= \pi_c\, p(x \mid y = c)$; posterior odds = prior odds $\times$ likelihood ratio; the likelihood splits by class; the MLE prior is the class frequency (Gibbs).
5. Gaussian classes: closed-form means and covariances (denominator N_c); LDA shares Σ and gets hyperplanes because $x^\top \Sigma^{-1} x$ cancels; QDA gets conics; a covariance costs $d(d+1)/2$; shrink when N_c is small.
6. Naive Bayes: independence given the class, additive score, one 1-D model per feature; ranking survives misspecification, calibration does not; smooth variances (λs_j^2) and counts (α).
7. Generative wins at small N if its assumptions are credible, discriminative wins at large N; the crossing of the two learning curves is the evidence; a flat floor is approximation error.
8. Imbalance is an estimation issue; prior shift is a modelling mismatch: swap priors (generative) or reweight the odds by π_d/π_t and $(1-\pi_d)/(1-\pi_t)$ (discriminative, needs calibration); resampling changes the effective prior.
9. Calibration is about what 0.8 means; the threshold is about what to do at 0.8. Correct the prior first, then threshold.
10. Confidence is not familiarity: add a likelihood-based reject option validated on held-out novelty; combine a discriminative ranker with a generative plausibility check; language models pair generative pretraining with a discriminative head.